

BACKEND DOCUMENTATION

=====

Purpose

This project's backend is the Python runtime that:

1. Captures frames from the camera.
2. Runs object detection with YOLO.
3. Optionally runs depth estimation with MiDaS.
4. Estimates obstacle distance and urgency.
5. Produces alerts, voice guidance, and assistant answers.
6. Streams everything to the frontend through FastAPI + WebSocket.

For backend work, the most important file is `server.py` - `/C:/MISSION_ASSIST/server.py`. The `src/` folder contains the domain modules that the server orchestrates.

Backend File Map

- `server.py` - `/C:/MISSION_ASSIST/server.py`: FastAPI app, WebSocket API, pipeline lifecycle, frame streaming.
- `src/detection.py` - `/C:/MISSION_ASSIST/src/detection.py`: YOLO wrapper and Detection model.
- `src/navigation.py` - `/C:/MISSION_ASSIST/src/navigation.py`: converts detections + depth into user-facing advice.
- `src/ranging.py` - `/C:/MISSION_ASSIST/src/ranging.py`: approximate monocular distance estimation and smoothing.
- `src/depth.py` - `/C:/MISSION_ASSIST/src/depth.py`: MiDaS depth model loading and cached inference.
- `src/voice.py` - `/C:/MISSION_ASSIST/src/voice.py`: thread-safe text-to-speech queue.
- `src/alerts.py` - `/C:/MISSION_ASSIST/src/alerts.py`: alert repetition suppression.
- `src/assistant_llm.py` - `/C:/MISSION_ASSIST/src/assistant_llm.py`: scene-to-prompt conversion and Gemini HTTP call.
- `src/speech_input.py` - `/C:/MISSION_ASSIST/src/speech_input.py`: Windows speech recognition helper for the GUI.

Architecture

Browser React UI

|
| WebSocket JSON messages
v

FastAPI server (`server.py`)

|
| starts/stops
v

Pipeline thread

|
+--> YOLO detector
+--> MiDaS depth estimator (upgraded mode only)
+--> Navigation engine

```

+--> Distance estimator + smoother
+--> Alert suppressor
+--> Voice engine
|
v
Events pushed into asyncio queue
|
v
WebSocket sender task
|
v
Frontend updates video, detections, stats, alerts

```

Runtime Lifecycle

1. Server startup:

When `server.py - /C:/MISSION_ASSIST/server.py` is run directly, it starts Uvicorn on port 8000 at `server.py:426 - /C:/MISSION_ASSIST/server.py#L426`.

2. WebSocket connection:

When the frontend connects to `/ws`, the backend accepts the socket, creates a fresh asyncio queue, and launches a sender coroutine that forwards pipeline events to the client at `server.py:356 - /C:/MISSION_ASSIST/server.py#L356`.

3. Pipeline start:

When the client sends:

```
{"type": "start", "mode": "basic", "confidence": 0.6, "alert_delay": 1.5}
```

the WebSocket handler calls `pipeline.start(config, loop)` at `server.py:381 - /C:/MISSION_ASSIST/server.py#L381`.

4. Background processing loop:

The pipeline thread:

1. Loads voice.
2. Loads YOLO.
3. Optionally loads MiDaS.
4. Optionally starts an ultrasonic sensor.
5. Opens the camera.
6. Loops over frames until stopped.

This happens inside `Pipeline._run()` starting at `server.py:117 - /C:/MISSION_ASSIST/server.py#L117`.

5. Event fan-out:

Each loop iteration can emit:

- frame
- detections
- alert
- stats
- distance
- error
- status

Events are added to the asyncio queue through `_push()` at `server.py:109` - `/C:/MISSION_ASSIST/server.py#L109`.

6. Stop / disconnect:

Stopping or disconnecting sets `_running = False`, stops voice, releases the camera, and emits `{"type": "status", "running": false}` at `server.py:89` - `/C:/MISSION_ASSIST/server.py#L89` and `server.py:419` - `/C:/MISSION_ASSIST/server.py#L419`.

Data Contracts

Client to server:

Defined in the docstring at `server.py:25` - `/C:/MISSION_ASSIST/server.py#L25`:

- start
- stop
- settings
- depth_overlay
- ask

Server to client:

Defined in the docstring at `server.py:14` - `/C:/MISSION_ASSIST/server.py#L14`:

- frame
- detections
- alert
- stats
- distance
- assistant
- error
- status

Detailed Walkthrough

1. server.py

What this file does:

This file is both:

1. The API server.
2. The application orchestrator.

It does not implement the detection or navigation algorithms itself. Instead, it wires together the modules in src/.

Line-by-line explanation:

Header and imports

- server.py:1 - /C:/VISSION_ASSIST/server.py#L1 starts a long module docstring that explains the server role and message format.
- server.py:14 - /C:/VISSION_ASSIST/server.py#L14 documents outbound messages.
- server.py:25 - /C:/VISSION_ASSIST/server.py#L25 documents inbound messages.
- server.py:42 - /C:/VISSION_ASSIST/server.py#L42 imports standard library modules for async work, encoding, timing, and threading.
- server.py:47 - /C:/VISSION_ASSIST/server.py#L47 builds the absolute src/ path.
- server.py:48 - /C:/VISSION_ASSIST/server.py#L48 injects src/ into sys.path so local modules like detection and navigation can be imported directly.
- server.py:51 - /C:/VISSION_ASSIST/server.py#L51 imports OpenCV, NumPy, and Torch.
- server.py:55 - /C:/VISSION_ASSIST/server.py#L55 imports FastAPI and WebSocket classes.
- server.py:58 - /C:/VISSION_ASSIST/server.py#L58 imports Uvicorn for running the HTTP server.

Pipeline class state

- server.py:63 - /C:/VISSION_ASSIST/server.py#L63 defines Pipeline, the core runtime controller.
- server.py:67 - /C:/VISSION_ASSIST/server.py#L67 begins the constructor.
- server.py:68 - /C:/VISSION_ASSIST/server.py#L68 stores the worker thread handle.
- server.py:69 - /C:/VISSION_ASSIST/server.py#L69 tracks whether the loop should keep running.
- server.py:70 - /C:/VISSION_ASSIST/server.py#L70 stores runtime config sent by the client.
- server.py:71 - /C:/VISSION_ASSIST/server.py#L71 tracks whether depth overlay should be drawn in rendered frames.
- server.py:72 - /C:/VISSION_ASSIST/server.py#L72 creates an asyncio queue used to move events from the worker thread to the WebSocket task.
- server.py:73 - /C:/VISSION_ASSIST/server.py#L73 stores the owning event loop reference.
- server.py:74 - /C:/VISSION_ASSIST/server.py#L74 holds the TTS engine instance.
- server.py:75 - /C:/VISSION_ASSIST/server.py#L75 tracks the last alert timestamp for cooldown behavior.
- server.py:76 - /C:/VISSION_ASSIST/server.py#L76 caches the most recent detections for LLM questions.
- server.py:77 - /C:/VISSION_ASSIST/server.py#L77 caches the last sonar distance.
- server.py:78 - /C:/VISSION_ASSIST/server.py#L78 records whether depth has successfully produced any map yet.

Pipeline control methods

- server.py:80 - /C:/VISSION_ASSIST/server.py#L80 defines start().
- server.py:81 - /C:/VISSION_ASSIST/server.py#L81 prevents double-starting the pipeline.
- server.py:83 - /C:/VISSION_ASSIST/server.py#L83 stores the new config.

- server.py:84 - /C:/VISSION_ASSIST/server.py#L84 stores the main asyncio loop so thread-safe queue writes are possible.
- server.py:85 - /C:/VISSION_ASSIST/server.py#L85 marks the pipeline as running.
- server.py:86 - /C:/VISSION_ASSIST/server.py#L86 creates a daemon thread targeting `_run`.
- server.py:87 - /C:/VISSION_ASSIST/server.py#L87 starts the thread.

- server.py:89 - /C:/VISSION_ASSIST/server.py#L89 defines `stop()`.
- server.py:90 - /C:/VISSION_ASSIST/server.py#L90 tells the main loop to exit.
- server.py:91 - /C:/VISSION_ASSIST/server.py#L91 clears cached detections.
- server.py:92 - /C:/VISSION_ASSIST/server.py#L92 clears cached distance.
- server.py:93 - /C:/VISSION_ASSIST/server.py#L93 resets depth state.
- server.py:94 - /C:/VISSION_ASSIST/server.py#L94 checks whether voice exists.
- server.py:95 - /C:/VISSION_ASSIST/server.py#L95 attempts to stop voice cleanly.

- server.py:98 - /C:/VISSION_ASSIST/server.py#L98 defines `update_settings()`.
- server.py:99 - /C:/VISSION_ASSIST/server.py#L99 merges partial config from the client into the current config.
- server.py:101 - /C:/VISSION_ASSIST/server.py#L101 resets the cooldown so new settings apply immediately.

- server.py:103 - /C:/VISSION_ASSIST/server.py#L103 toggles depth overlay rendering.
- server.py:106 - /C:/VISSION_ASSIST/server.py#L106 explicitly resets the alert timer.

Queue bridge

- server.py:110 - /C:/VISSION_ASSIST/server.py#L110 defines `_push()`.
- server.py:111 - /C:/VISSION_ASSIST/server.py#L111 ensures the main event loop still exists.
- server.py:112 - /C:/VISSION_ASSIST/server.py#L112 uses `asyncio.run_coroutine_threadsafe(...)` because the worker thread cannot await directly.
- server.py:113 - /C:/VISSION_ASSIST/server.py#L113 schedules `queue.put(event)` on the main loop.

Pipeline bootstrap inside `_run()`

- server.py:117 - /C:/VISSION_ASSIST/server.py#L117 begins the background pipeline.
- server.py:118 - /C:/VISSION_ASSIST/server.py#L118 imports `ObjectDetector` lazily, so the server can import faster before the pipeline starts.
- server.py:119 - /C:/VISSION_ASSIST/server.py#L119 imports `Navigator`.
- server.py:120 - /C:/VISSION_ASSIST/server.py#L120 imports voice engine and priority constants.
- server.py:121 - /C:/VISSION_ASSIST/server.py#L121 imports `AlertSuppressor`.
- server.py:123 - /C:/VISSION_ASSIST/server.py#L123 chooses GPU if CUDA is available, otherwise CPU.
- server.py:124 - /C:/VISSION_ASSIST/server.py#L124 builds a frontend-friendly device label string.
- server.py:127 - /C:/VISSION_ASSIST/server.py#L127 creates the voice engine.
- server.py:128 - /C:/VISSION_ASSIST/server.py#L128 starts the voice worker thread.
- server.py:131 - /C:/VISSION_ASSIST/server.py#L131 reads the requested mode from config.
- server.py:132 - /C:/VISSION_ASSIST/server.py#L132 maps mode to YOLO weights.
- server.py:133 - /C:/VISSION_ASSIST/server.py#L133 resolves the model path.
- server.py:135 - /C:/VISSION_ASSIST/server.py#L135 begins safe YOLO loading.

- server.py:136 - /C:/VISSION_ASSIST/server.py#L136 creates ObjectDetector.
- server.py:137 - /C:/VISSION_ASSIST/server.py#L137 passes the current confidence threshold.
- server.py:138 - /C:/VISSION_ASSIST/server.py#L138 forces detector device alignment with the pipeline.
- server.py:139 - /C:/VISSION_ASSIST/server.py#L139 catches model load failure.
- server.py:140 - /C:/VISSION_ASSIST/server.py#L140 sends the error to the client.
- server.py:141 - /C:/VISSION_ASSIST/server.py#L141 stops voice because the pipeline cannot proceed.
- server.py:142 - /C:/VISSION_ASSIST/server.py#L142 marks the pipeline as not running.
- server.py:143 - /C:/VISSION_ASSIST/server.py#L143 exits the thread early.
- server.py:145 - /C:/VISSION_ASSIST/server.py#L145 creates the Navigator.

Optional depth and ultrasonic setup

- server.py:148 - /C:/VISSION_ASSIST/server.py#L148 initializes depth_est to None.
- server.py:150 - /C:/VISSION_ASSIST/server.py#L150 only enables MiDaS in upgraded mode.
- server.py:152 - /C:/VISSION_ASSIST/server.py#L152 imports DepthEstimator lazily.
- server.py:153 - /C:/VISSION_ASSIST/server.py#L153 creates the depth estimator with frame skipping.
- server.py:155 - /C:/VISSION_ASSIST/server.py#L155 catches MiDaS failure and reports it without crashing the whole pipeline.
- server.py:159 - /C:/VISSION_ASSIST/server.py#L159 initializes the optional ultrasonic sensor reference.
- server.py:160 - /C:/VISSION_ASSIST/server.py#L160 only tries sonar when enabled and a port exists.
- server.py:162 - /C:/VISSION_ASSIST/server.py#L162 imports UltrasonicSensor.
- server.py:163 - /C:/VISSION_ASSIST/server.py#L163 constructs the sensor.
- server.py:167 - /C:/VISSION_ASSIST/server.py#L167 starts reading from the sensor.
- server.py:168 - /C:/VISSION_ASSIST/server.py#L168 treats sonar startup as optional by sending an error instead of aborting.

Camera and per-run state

- server.py:172 - /C:/VISSION_ASSIST/server.py#L172 opens camera index 0.
- server.py:173 - /C:/VISSION_ASSIST/server.py#L173 fixes width to 640.
- server.py:174 - /C:/VISSION_ASSIST/server.py#L174 fixes height to 480.
- server.py:175 - /C:/VISSION_ASSIST/server.py#L175 validates that the camera opened successfully.
- server.py:176 - /C:/VISSION_ASSIST/server.py#L176 notifies the frontend if camera access fails.
- server.py:177 - /C:/VISSION_ASSIST/server.py#L177 stops voice on failure.
- server.py:181 - /C:/VISSION_ASSIST/server.py#L181 resets alert timing for a fresh session.
- server.py:182 - /C:/VISSION_ASSIST/server.py#L182 creates the duplicate-alert guard.
- server.py:183 - /C:/VISSION_ASSIST/server.py#L183 scales repeat suppression based on alert_delay.
- server.py:185 - /C:/VISSION_ASSIST/server.py#L185 announces that processing has started.
- server.py:187 - /C:/VISSION_ASSIST/server.py#L187 creates an FPS rolling buffer.
- server.py:188 - /C:/VISSION_ASSIST/server.py#L188 saves the previous timestamp for FPS calculation.
- server.py:190 - /C:/VISSION_ASSIST/server.py#L190 maps navigation urgencies to voice priorities.

Main frame loop

- server.py:192 - /C:/VISSION_ASSIST/server.py#L192 starts the processing loop.
- server.py:193 - /C:/VISSION_ASSIST/server.py#L193 reads a frame from the camera.
- server.py:194 - /C:/VISSION_ASSIST/server.py#L194 handles temporary capture failures.
- server.py:195 - /C:/VISSION_ASSIST/server.py#L195 sleeps briefly to avoid a hot spin loop.
- server.py:198 - /C:/VISSION_ASSIST/server.py#L198 captures the current time.
- server.py:199 - /C:/VISSION_ASSIST/server.py#L199 computes instantaneous FPS and stores it in the rolling buffer.
- server.py:200 - /C:/VISSION_ASSIST/server.py#L200 updates the previous timestamp.
- server.py:202 - /C:/VISSION_ASSIST/server.py#L202 normalizes frame size before processing.
- server.py:205 - /C:/VISSION_ASSIST/server.py#L205 updates depth or returns cached depth if the frame is skipped.
- server.py:206 - /C:/VISSION_ASSIST/server.py#L206 exposes whether depth has ever become ready.
- server.py:209 - /C:/VISSION_ASSIST/server.py#L209 applies live confidence changes to the detector.
- server.py:210 - /C:/VISSION_ASSIST/server.py#L210 runs YOLO detection.
- server.py:211 - /C:/VISSION_ASSIST/server.py#L211 caches detections for AI assistant queries.
- server.py:214 - /C:/VISSION_ASSIST/server.py#L214 uses depth-aware analysis when depth exists.
- server.py:217 - /C:/VISSION_ASSIST/server.py#L217 falls back to area-based analysis when depth is unavailable.

Sonar fusion logic

- server.py:220 - /C:/VISSION_ASSIST/server.py#L220 reads the ultrasonic distance if a sensor is active.
- server.py:221 - /C:/VISSION_ASSIST/server.py#L221 caches it for the assistant.
- server.py:223 - /C:/VISSION_ASSIST/server.py#L223 imports sonar thresholds.
- server.py:224 - /C:/VISSION_ASSIST/server.py#L224 classifies the reading into danger, warning, or safe.
- server.py:227 - /C:/VISSION_ASSIST/server.py#L227 emits the distance event to the frontend.
- server.py:229 - /C:/VISSION_ASSIST/server.py#L229 produces a hard-stop alert when sonar sees something very close and vision did not.
- server.py:233 - /C:/VISSION_ASSIST/server.py#L233 enforces the alert cooldown.
- server.py:234 - /C:/VISSION_ASSIST/server.py#L234 checks with AlertSuppressor before repeating the alert.
- server.py:237 - /C:/VISSION_ASSIST/server.py#L237 speaks the message at critical priority.
- server.py:238 - /C:/VISSION_ASSIST/server.py#L238 sends the same alert to the frontend.

Navigation alert logic

- server.py:242 - /C:/VISSION_ASSIST/server.py#L242 gets the current timestamp for alerting.
- server.py:244 - /C:/VISSION_ASSIST/server.py#L244 requires actual advice from the navigator.
- server.py:245 - /C:/VISSION_ASSIST/server.py#L245 applies the configured cooldown.
- server.py:246 - /C:/VISSION_ASSIST/server.py#L246 to server.py:251 - /C:/VISSION_ASSIST/server.py#L251 decide whether the alert should be emitted.
- server.py:253 - /C:/VISSION_ASSIST/server.py#L253 checks whether voice is enabled.
- server.py:254 - /C:/VISSION_ASSIST/server.py#L254 speaks the message using mapped priority.

- server.py:256 - /C:/VISSION_ASSIST/server.py#L256 emits the alert JSON to the frontend.

Frame encoding and telemetry

- server.py:260 - /C:/VISSION_ASSIST/server.py#L260 renders overlays on top of the frame.
- server.py:263 - /C:/VISSION_ASSIST/server.py#L263 JPEG-encodes the image.
- server.py:264 - /C:/VISSION_ASSIST/server.py#L264 base64-encodes the bytes for JSON transport.
- server.py:265 - /C:/VISSION_ASSIST/server.py#L265 pushes the frame event.
- server.py:268 - /C:/VISSION_ASSIST/server.py#L268 to server.py:280 - /C:/VISSION_ASSIST/server.py#L280 serialize detection summaries for the UI.
- server.py:283 - /C:/VISSION_ASSIST/server.py#L283 to server.py:288 - /C:/VISSION_ASSIST/server.py#L288 emit stats.
- server.py:290 - /C:/VISSION_ASSIST/server.py#L290 releases the camera.
- server.py:291 - /C:/VISSION_ASSIST/server.py#L291 stops sonar.
- server.py:292 - /C:/VISSION_ASSIST/server.py#L292 stops voice.
- server.py:293 - /C:/VISSION_ASSIST/server.py#L293 sends a stopped status event.

Rendering layer

- server.py:295 - /C:/VISSION_ASSIST/server.py#L295 defines `_render()`, the final annotation step before streaming.
- server.py:299 - /C:/VISSION_ASSIST/server.py#L299 overlays the MiDaS heatmap.
- server.py:304 - /C:/VISSION_ASSIST/server.py#L304 loops through detections to draw bounding boxes.
- server.py:310 - /C:/VISSION_ASSIST/server.py#L310 colors boxes by estimated distance when centimeters are available.
- server.py:315 - /C:/VISSION_ASSIST/server.py#L315 otherwise colors boxes by relative depth.
- server.py:330 - /C:/VISSION_ASSIST/server.py#L330 draws left/center/right guide lines.
- server.py:333 - /C:/VISSION_ASSIST/server.py#L333 draws the sonar HUD when available.

FastAPI and WebSocket API

- server.py:344 - /C:/VISSION_ASSIST/server.py#L344 creates the FastAPI app.
- server.py:345 - /C:/VISSION_ASSIST/server.py#L345 creates one global Pipeline.
- server.py:349 - /C:/VISSION_ASSIST/server.py#L349 mounts the frontend as static files.
- server.py:351 - /C:/VISSION_ASSIST/server.py#L351 defines the root route.
- server.py:356 - /C:/VISSION_ASSIST/server.py#L356 defines the `/ws` WebSocket endpoint.
- server.py:360 - /C:/VISSION_ASSIST/server.py#L360 resets the queue for the current client session.
- server.py:363 - /C:/VISSION_ASSIST/server.py#L363 creates a sender coroutine that forwards pipeline events.
- server.py:377 - /C:/VISSION_ASSIST/server.py#L377 receives incoming WebSocket JSON from the frontend.
- server.py:381 - /C:/VISSION_ASSIST/server.py#L381 handles start.
- server.py:385 - /C:/VISSION_ASSIST/server.py#L385 handles stop.
- server.py:388 - /C:/VISSION_ASSIST/server.py#L388 handles settings.
- server.py:393 - /C:/VISSION_ASSIST/server.py#L393 handles `depth_overlay`.
- server.py:396 - /C:/VISSION_ASSIST/server.py#L396 handles ask.
- server.py:403 - /C:/VISSION_ASSIST/server.py#L403 runs the LLM request in a worker thread so the event loop does not block.
- server.py:411 - /C:/VISSION_ASSIST/server.py#L411 sends the assistant response back to the client.

- server.py:422 - /C:/VISSION_ASSIST/server.py#L422 stops the pipeline during cleanup.

End-to-End Data Flow

Flow 1: live vision stream:

1. The browser connects to /ws and sends a start message.
2. The backend stores the config and starts the background pipeline thread.
3. The pipeline opens the camera and reads frames continuously.
4. Each frame is resized to a standard 640x480.
5. YOLO detects objects in the frame.
6. If upgraded mode is enabled, MiDaS produces a relative depth map.
7. The navigator combines detections, depth, and ranging to decide the most important obstacle.
8. Alerts are filtered by cooldown and duplicate suppression.
9. The annotated frame is JPEG-encoded and base64-encoded.
10. The detection list, stats, alerts, and frame are pushed into the asyncio queue.
11. The WebSocket sender task forwards those events to the frontend.

Flow 2: assistant question:

1. The frontend sends {"type":"ask","question":"..."} over the same WebSocket.
2. The backend snapshots recent detections, last sonar distance, and depth readiness.
3. assistant_llm.py converts that scene state into a safety-oriented prompt.
4. Gemini is called over plain HTTP.
5. The answer is sent back as {"type":"assistant","answer":"..."}.
6. If voice is enabled, the same answer is spoken through VoiceEngine.

Flow 3: live settings update:

1. The frontend sends {"type":"settings", ... }.
2. The server merges those values into Pipeline._config.
3. The detector immediately reads the latest confidence threshold on the next frame.
4. Alert cooldown is reset so the user feels the new setting immediately.

API Reference

WebSocket endpoint:

- Endpoint: ws://<host>:8000/ws
- Transport: JSON text messages over WebSocket
- Primary direction: bidirectional event stream

Client -> server messages:

start

```
{  
  "type": "start",  
  "mode": "basic",  
  "confidence": 0.6,
```

```
"alert_delay": 1.5,  
"voice_enabled": true,  
"ultrasonic_enabled": false,  
"ultrasonic_port": "COM3",  
"ultrasonic_baud": 9600  
}
```

Purpose: starts the vision pipeline with the given runtime configuration.

stop

```
{"type":"stop"}
```

Purpose: stops the live pipeline and releases resources.

settings

```
{"type":"settings","confidence":0.65,"alert_delay":1.2}
```

Purpose: updates config live without recreating the whole backend process.

depth_overlay

```
{"type":"depth_overlay","enabled":true}
```

Purpose: toggles rendering of the MiDaS heatmap on outgoing frames.

ask

```
{"type":"ask","question":"What is ahead?","api_key":"<gemini-key>"}
```

Purpose: asks the assistant for a scene-aware natural-language answer.

Server -> client messages:

frame

```
{"type":"frame","data":"<base64-jpeg>"}
```

Purpose: carries the latest annotated camera frame.

detections

```
{
  "type": "detections",
  "data": [
    { "name": "person", "conf": 0.92, "depth": 0.78, "pos": "left", "area": 52200, "distance_cm": 95.0 }
  ]
}
```

Purpose: gives the frontend a compact summary of objects currently visible.

alert

```
{ "type": "alert", "message": "Warning! Person about 100 centimeters away on your left. Move right.", "urgency": "critical" }
```

Purpose: communicates the primary safety message to the UI.

stats

```
{ "type": "stats", "fps": 18.3, "device": "GPU (CUDA)", "depth_ready": true, "mode": "upgraded", "sensor_on": false }
```

Purpose: operational telemetry for the UI.

distance

```
{ "type": "distance", "cm": 84.2, "zone": "warning" }
```

Purpose: sonar-specific measurement event.

assistant

```
{ "type": "assistant", "answer": "There is a person about one meter ahead on your left." }
```

Purpose: delivers the LLM answer to the frontend.

status

```
{ "type": "status", "running": true }
```

Purpose: indicates whether the pipeline is active.

error

```
{"type":"error","message":"YOLO load failed: ..."}

```

Purpose: surfaces runtime failures without necessarily crashing the whole server process.

Backend Modules

2. detection.py

detection.py wraps Ultralytics YOLO and converts raw model output into the project-friendly Detection dataclass. The dataclass in src/detection.py:18 - /C:/MISSION_ASSIST/src/detection.py#L18 stores class name, confidence, box coordinates, and computed fields like center_x, center_y, and area. The detector class in src/detection.py:48 - /C:/MISSION_ASSIST/src/detection.py#L48 lazily imports heavy ML dependencies so the server can import faster and avoid loading YOLO before the user actually starts the pipeline. detect() at src/detection.py:82 - /C:/MISSION_ASSIST/src/detection.py#L82 runs the model, filters by confidence, and returns normalized Detection objects.

3. navigation.py

navigation.py is the decision layer. It decides which object matters most, where it is, how urgent it is, and what message should be spoken. In upgraded mode, analyse() at src/navigation.py:79 - /C:/MISSION_ASSIST/src/navigation.py#L79 reads both detections and MiDaS depth. It computes adaptive percentiles from the scene, samples depth around each object's center, estimates distance if possible, and then produces one NavigationAdvice. In basic mode, analyse_by_area() at src/navigation.py:147 - /C:/MISSION_ASSIST/src/navigation.py#L147 falls back to area and class priors. This module is the heart of the obstacle-prioritization logic.

4. ranging.py

ranging.py turns object size into approximate centimeters. DistanceEstimator at src/ranging.py:57 - /C:/MISSION_ASSIST/src/ranging.py#L57 uses assumed object dimensions plus camera field of view to estimate distance from bounding-box width or height. This is not true metric depth, but it is good enough to sort urgency. DistanceSmoother at src/ranging.py:148 - /C:/MISSION_ASSIST/src/ranging.py#L148 dampens frame-to-frame jumps so voice alerts do not oscillate.

5. depth.py

depth.py wraps MiDaS for monocular relative depth estimation. DepthEstimator at src/depth.py:30 - /C:/MISSION_ASSIST/src/depth.py#L30 chooses CPU or GPU, loads a fallback chain of models, and computes depth only every frame_skip frames. The output is normalized disparity, so higher values mean closer objects. This is why the heatmap and navigation logic interpret bright/hot regions as near obstacles.

6. voice.py

voice.py protects the system from speech overload. VoiceEngine at src/voice.py:43 - /C:/VISSION_ASSIST/src/voice.py#L43 keeps a bounded priority queue, deduplicates repeated speech, and lets critical messages preempt lower-priority ones. This is essential because the backend can generate alerts faster than a TTS engine can speak them.

7. alerts.py

alerts.py decides whether an alert is worth repeating. AlertSuppressor at src/alerts.py:18 - /C:/VISSION_ASSIST/src/alerts.py#L18 compares the new alert with the previous one, watches urgency changes, checks how much the distance changed, and blocks noisy repeats for a minimum interval.

8. assistant_llm.py

assistant_llm.py integrates the assistant. It builds a prompt from scene state, recent detections, and sonar distance, then calls Gemini through plain HTTP. It does not require a heavyweight SDK, which keeps the backend easier to understand and deploy.

9. speech_input.py

speech_input.py is a desktop helper for capturing spoken user questions through Windows PowerShell and System.Speech. It is not part of the WebSocket pipeline, but it supports the overall backend capability for voice interaction.

Backend Tech Stack And Why It Is Used

FastAPI:

Used in server.py - /C:/VISSION_ASSIST/server.py for the HTTP server and WebSocket endpoint. It is a strong fit because:

- it gives async networking support out of the box,
- WebSocket support is simple,
- the code stays compact,
- it is easy to serve both API and static frontend content from one process.

Uvicorn:

Used to run the ASGI app. It is lightweight, fast, and the default operational partner for FastAPI.

WebSocket:

Used instead of plain REST polling because the frontend needs a continuous stream of frames, detections, stats, and alerts. This backend is event-driven, so WebSocket is the natural transport.

OpenCV:

Used for camera capture, frame resizing, drawing overlays, JPEG encoding, and color-map visualization. It is the practical computer-vision utility layer around the ML models.

PyTorch:

Used because both YOLO and MiDaS depend on Torch in this project. It also provides device management for CPU/GPU and tensor operations for inference.

Ultralytics YOLO:

Used for object detection because it gives fast pretrained detection with minimal wrapper code. The backend needs class labels and bounding boxes in real time, and YOLO is doing that job here.

MiDaS:

Used for relative depth estimation from a single camera. Without MiDaS, the backend knows only what object is present, not which visible object is closest in the scene. MiDaS improves prioritization.

pyttsx3:

Used for offline text-to-speech. That matters here because safety guidance should still work locally even without internet access.

Gemini HTTP API:

Used for scene-aware assistant answers. The project uses plain HTTP instead of a large SDK so the integration stays readable and dependency-light.

PowerShell System.Speech:

Used on Windows for speech-to-text capture without requiring a separate cloud STT provider.

Concurrency Model

The backend uses three concurrency layers:

1. FastAPI's asyncio event loop for network traffic.
2. One background thread for the vision pipeline.
3. Additional short-lived threads for voice output and blocking assistant work.

This design was likely chosen because the camera and ML pipeline are long-running and blocking, while the WebSocket server must stay responsive. The background thread isolates the vision loop from the async server.

State Management

Important runtime state lives in Pipeline:

- `_running`: whether the loop should continue.
- `_config`: live runtime configuration from the client.
- `_last_alert`: cooldown marker.

- `_last_detections`: cached for assistant context.
- `_last_distance_cm`: cached sonar value.
- `_depth_ready`: whether MiDaS has successfully produced at least one map.

The backend is stateful. It is not a stateless request-per-response API server.

Operational Notes

Startup cost:

Model loading happens when the pipeline starts, not at process boot. This keeps initial import lighter, but it makes the first start slower.

Single-client orientation:

The current design creates one global Pipeline and swaps its queue per WebSocket session. That means this backend is best understood as a single-user live runtime, not a multi-tenant backend service.

Failure behavior:

The backend often degrades gracefully:

- if MiDaS fails, it can continue without depth,
- if the ultrasonic sensor fails, vision can still run,
- if the assistant fails, the main detection loop still works.

Risks And Gaps

Missing ultrasonic module:

`server.py:162` - `/C:/VISSION_ASSIST/server.py#L162` imports ultrasonic, but there is no matching file in the current workspace snapshot. That makes ultrasonic support incomplete here.

Case-sensitive deployment risk:

`server.py:348` - `/C:/VISSION_ASSIST/server.py#L348` points to frontend, while the folder in this workspace is `FRONTEND`. Windows usually tolerates that, Linux usually does not.

Heavy runtime responsibilities in one file:

`server.py` currently owns transport, pipeline orchestration, rendering, and session state. It works, but for a larger production backend you would usually split transport, service orchestration, and rendering into separate modules.

Suggested Reading Order

1. `server.py` - `/C:/VISSION_ASSIST/server.py`
2. `src/detection.py` - `/C:/VISSION_ASSIST/src/detection.py`
3. `src/navigation.py` - `/C:/VISSION_ASSIST/src/navigation.py`
4. `src/ranging.py` - `/C:/VISSION_ASSIST/src/ranging.py`

5. src/depth.py - /C:/MISSION_ASSIST/src/depth.py
6. src/voice.py - /C:/MISSION_ASSIST/src/voice.py
7. src/alerts.py - /C:/MISSION_ASSIST/src/alerts.py
8. src/assistant_llm.py - /C:/MISSION_ASSIST/src/assistant_llm.py

Final Summary

This backend is a real-time, stateful, event-driven Python system. FastAPI and WebSocket provide the transport. A background pipeline thread handles camera frames and model inference. YOLO detects objects, MiDaS estimates relative closeness, ranging approximates centimeters, navigation decides what matters, alerts prevents repetition, voice speaks the result, and the assistant explains the scene in natural language. The design is practical for a live assistive-vision application because it prioritizes responsiveness, local inference, graceful fallback, and continuous streaming over purely stateless API patterns.